

3장 데스크톱 버전 검토 의견 v2

대상 문서: 노션 「3장 클로드 코드 시작하기 (손글씨 인식 앱) 데스크톱 버전」 (2026-08-08 04:10 UTC 판)
검토 관점: 2026년 9월부터 수업을 듣는 대학 1학년이 혼자 따라 할 때 어디서 막히고 무엇을 오해하는가
검토 범위: 본문 전체(그림 주소를 뺀 4만 자)와 삽입된 그림 118장 전부
이전 검토서: [2026-08-07-3장-데스크톱-버전-검토의견.md](#) — A 5건, B 7건, C 5건

한눈에

A 등급 5건은 모두 해소됐다. 3.1 설치에서 학생이 멈추던 문제(Homebrew, gh)가 없어졌고, 환경 의존성과 SSL 오류도 본문으로 올라왔으며, 3.4 핵심 정리는 교재와 수업의 차이를 밝힌 뒤 실제 실습 내용으로 교체됐다. 첫 90분 이탈 위험은 사라졌다고 본다.

B, C 등급 12건은 아직 반영되지 않았다. 이번 판에서 섹션마다 번호가 붙었으므로, 아래에 번호와 검색어를 붙여 다시 신는다.

새로 확인한 것이 22건이다. 그중 둘은 A 등급이다. 하나는 명령에서 공백이 빠져 그대로는 실행되지 않고, 하나는 같은 절 안의 두 프롬프트가 서로 어긋난다.

구분	건수
이전 A 등급	5건 → 전부 해소
이전 B 등급	7건 → 0건 해소
이전 C 등급	5건 → 0건 해소 (둘은 범위가 더 넓어짐)
새 A 등급	2건
새 B 등급	10건
새 C 등급	10건

읽는 법

심각도는 학생이 실제로 막히는가를 기준으로 매겼다. 이전 검토서와 같은 기준이다.

등급	뜻
A	여기서 멈춘다. 다음 단계로 못 간다
B	진행은 되지만 틀리게 이해하거나, 화면이 달라 불안해한다

등급	뜻
C	다듬으면 좋다

지적마다 위치(토글 경로)와 🔍 검색어를 붙였다. 노선 토글이 5단계까지 중첩돼 있고 접힌 상태가 기본이라, 검색어로 찾아가는 편이 빠르다. 검색어는 문서에서 되도록 한 번만 나오는 문구로 골랐다.

그림 번호는 이번 판 기준으로 다시 매겼다. 이전 검토서와 번호가 다르다. 그림 051부터는 이전 번호에 3을 더한 값이다 (이전 048 → 이번 051).

1. 이전 17건 대조

A 등급 — 5건 전부 해소

이전	내용	판정	어디에 반영됐나
A-1	Homebrew 설치 단계 없음	해소	3.1.1.1 ▶ b) Homebrew 확인 및 설치 방법 신설. 확인 명령, 설치 명령, Next steps: 세 줄, M칩과 인텔 구분까지 전부 들어갔다. 윈도우도 3.1.1.2 ▶ c) 에 같은 구조로 들어갔다
A-2	gh 설치 단계 없음	해소	3.3.3.8 ▶ a.1) GitHub CLI ... gh 설치 신설. 맥, 윈도우 둘 다
A-3	실습 결과가 환경에 따라 달라진다는 사실이 없음	해소	3.2.2.2 ▶ f.1.4) PyTorch를 채택해서 진행한 이유 와 f.2.2) 실습에서 사용할 프롬프트 신설. 다만 새 문제 두 가지가 생겼다 → A-신1, B-신3
A-4	SSL 인증서 오류가 그림 안에만 있음	해소	f.2.4) MNIST 다운로드 과정에서 SSL: CERTIFICATE_VERIFY_FAILED 오류가 발생하는 경우 신설
A-5	핵심 정리가 교재 기준이라 수업과 다름	해소	3.4.1 에서 노드JS, 배치 파일, # 키 세 가지가 왜 다른지 밝히고, 3.4.2 에 수업에서 실제로 한 8개 키워드를 새로 실었다. 남은 것 하나 → C-신10

A-5 의 처리 방식이 특히 좋다. 교재 그림(그림 097)을 지우지 않고 남긴 채 "무엇이 왜 다른지"를 옆에 붙였다. 도구가 계속 바뀐다는 것 자체가 가르칠 내용이 된다는 취지가 그대로 살았다.

B, C 등급 — 12건 미반영

아래 2절, 3절에 섹션 번호를 붙여 다시 심는다.

2. A 등급 — 학생이 여기서 멈춘다 (새로 확인한 2건)

A-신1. 세션 시작 프롬프트와 실습 프롬프트가 서로 다르다

📍 위치 — 3.2 손글씨 인식 프로그램 (데스크톱 버전) ▶ 3.2.2 클로드 코드를 활용한 인식기 생성 ▶ 3.2.2.2 클로드 코드 세션 시작하고 프롬프트 입력하여 손글씨 인식 앱 작성 ▶ a) 세션 시작 및 프롬프트 입력

🔍 검색어 — 세션을 준비하고, 아래 프롬프트 입력

고칠 곳 — a) 안의 코드 블록

이 절을 순서대로 따라 하는 학생은 a) 에서 프롬프트를 먼저 입력한다. 그 프롬프트는 이것이다.

손글씨로 숫자를 입력하면 이것을 인식하는 코드를 만들어서 실행해 줘. 모든 코드와 주석을 한글로 작성해 줘.

PyTorch 지정이 없다. PyTorch 를 지정한 프롬프트는 한참 뒤 f.2.2) 실습에서 사용할 프롬프트 에 나온다. 그런데 f 는 a 보다 다섯 단계 뒤고, 그 사이에 b) ~ e) 와 f.1) 이 있다.

즉 문서 구조대로 따라가면 학생은 이미 잘못된 프롬프트를 보내 놓은 뒤에야 올바른 프롬프트를 만난다. 그 시점에는 클로드가 이미 Keras 로 코드를 만들었을 수 있고, 그러면 mnist_cnn.pt 가 안 생겨 3.3 웹 버전 실습이 통째로 어긋난다. A-3 을 고치면서 세운 장치가 순서 때문에 작동하지 않는 상태다.

f.2.1) 강의 노트에서 사용한 프롬프트 와 f.2.2) 실습에서 사용할 프롬프트 로 구분한 것은 좋은 처리다. 문제는 a) 가 그 구분을 모른 채 f.2.1 쪽을 그대로 싣고 있다는 점이다.

고칠 것 — 두 가지 중 하나면 된다.

1. a) 의 프롬프트를 f.2.2 의 것으로 바꾼다. 가장 간단하다.
2. a) 를 "프롬프트는 f.2.2 를 쓴다" 로 바꾸고 코드 블록을 지운다.

1번을 권한다. 아래를 그대로 옮겨 쓸 수 있다.

아래 프롬프트를 입력한다. (강의 노트를 만들 때 쓴 프롬프트와 다르다. 이유는 f.1.4 에 있다.)

손글씨로 숫자를 입력하면 이것을 인식하는 코드를 만들어서 실행해 줘.
PyTorch 를 사용하고, 학습된 가중치는 mnist_cnn.pt 로 저장해 줘.
모든 코드와 주석을 한글로 작성해 줘.

덧붙여, f.1) TensorFlow vs. PyTorch 가 f) 아래 있는 것도 순서를 어렵게 만든다. 환경 확인과 프롬프트 결정은 실습을 시작하기 전에 알아야 하는 내용이므로 a) 앞으로 올리는 편이 낫다.

A-신2. winget 설치 명령에서 공백이 빠져 그대로는 실행되지 않는다

📌 위치 — 3.1 클로드 데스크톱 설치 ▶ 3.1.1 클로드 데스크톱 앱 설치 및 실행 ▶ 3.1.1.2 (winget 활용한) 윈도우 설치 방법 및 실행 ▶ c) winget 확인 및 설치 방법 ▶ 방법 2 — 스토어를 쓸 수 없는 경우

🔍 검색어 — aka.ms/getwinget

고칠 곳 — 첫 번째 코드 블록

현재 이렇게 적혀 있다.

```
Invoke-WebRequest -Uri https://aka.ms/getwinget-OutFile winget.msixbundle
```

getwinget 과 **-OutFile** 사이의 공백이 없다. 이대로 실행하면 PowerShell 이 `https://aka.ms/getwinget-OutFile` 을 통째로 주소로 읽고, `winget.msixbundle` 은 정체불명의 인자가 된다. 파일이 안 받아지므로 다음 줄의 `Add-AppxPackage` 도 실패한다.

이 명령을 만나는 학생은 스토어가 막힌 학교 실습실 PC 를 쓰는 학생이다. 이미 한 번 막힌 뒤에 온 사람이라, 여기서 또 막히면 그날 실습을 포기한다.

고칠 것 — 공백을 넣는다.

```
Invoke-WebRequest -Uri https://aka.ms/getwinget -OutFile winget.msixbundle
```

노션에서 코드 블록을 편집할 때 줄이 접히면서 공백이 사라진 것으로 보인다. 같은 절의 다른 코드 블록도 한 번씩 확인하 시기를 권한다.

3. B 등급 — 진행은 되지만 오해하거나 불안해한다

3.1 이전 검토회의 B 등급 7건 (전부 미반영)

B-1. 부동산수 오차 설명의 예시가 실제로는 오차가 나지 않는다 (0/전 B-1)

📌 위치 ① — 3.3 CLAUDE.md 활용 (웹 버전) ▶ 3.3.3 숫자 인식 앱을 웹 버전으로도 작성 ▶ 3.3.3.7 클로드 의 구현 상황 중간 보고 ▶ a) 총 7개 태스크 중 4개 태스크 완료되었고, 5번 태스크 수행 중 치명적 오류 1건 발견했 다는 ... (그림 093)

📌 위치 ② — 같은 3.3.3.7 ▶ b) 구현 완료했고, 깃허브 메인에 병합하여 검증 진행 (그림 095)

🔍 검색어 — 180.00000000000003

고칠 곳 — 두 그림 안의 문장. 그림을 다시 만들거나, 본문에 정정 문구를 덧붙인다

두 그림 모두 이렇게 적혀 있다.

면적 평균 축소에서 $20 * (180/20)$ 이 180.00000000000003 이 되는 부동소수 오차 때문에...

틀렸다. 다시 확인했고, 실제로는 정확히 **180.0** 이다.

```
>>> 20 * (180/20)
180.0
```

호기심 있는 학생이 파이썬에 쳐 보면 오차가 안 난다. 그러면 설명 전체를 못 믿게 된다. 이 대목은 "검증을 통과했다고 옳은 것이 아니다" 라는 이 장의 가장 값진 교훈을 떠받치는 근거라서, 근거가 무너지면 교훈도 같이 무너진다.

오차가 실제로 나는 값은 이것이다. 저장소(`web_version/CLAUDE.md`)는 이미 정정돼 있고 노션 문서에만 옛 설명이 남아 있다.

```
>>> 19 * (21/19)
21.000000000000004
```

고칠 것 — 잘라낸 너비 21 을 새 너비 19 로 줄이는 경우로 예시를 바꾼다. 이 값이 마지막 열에서 인덱스를 하나 넘겨 실제로 `undefined` → `NaN` 을 만든다.

그림을 다시 만들기 어렵다면, 두 토글 안 본문에 한 줄만 덧붙여도 된다.

(정정) 위 화면의 $20 * (180/20)$ 은 실제로는 오차가 나지 않는다. 실제로 오차가 나는 것은 $19 * (21/19) = 21.000000000000004$ 이고, 이 값이 배열 밖을 읽게 만든다.

B-2. 저장소 이름이 본문과 화면에서 다르다 (이전 B-2)

📍 위치 ① — 3.3 ▶ 3.3.3 ▶ 3.3.3.8 깃허브에 올리기 및 웹 주소로 배포하기 ▶ a) 깃허브에 올리기 ▶ **a.4)** 클로드 대화창에서 다음과 같이 요청

📍 위치 ② — 같은 a.5) `git init`, 커밋, 저장소 생성, 푸시 개념 이해하기 ▶ **a.5.3)** 저장소 생성 — 깃허브에 빈 앨범을 만든다

🔍 검색어 — `my-first-app`

고칠 곳 — 두 곳의 코드 블록

a.4) 의 실습 프롬프트는 여전히 이렇다.

이 폴더를 깃허브 저장소로 만들어서 올려 줘. 저장소 이름은 `my-first-app`, 공개로.

그런데 바로 다음에 나오는 실제 화면(그림 074)의 프롬프트는 깃허브에 **Study01_MNIST_mac** 저장소를 만들고 작업 폴더의 내용을 올려줘. 이고, 문서 뒤쪽의 결과 링크도 전부 `Study01_MNIST_mac` 이다.

그대로 따라 한 학생은 `my-first-app` 이라는 저장소를 만든다. 그러면 뒤에 나오는 배포 주소도, 3.3.4 의 최종 결과 링크도 자기 것과 안 맞는다.

a.5.3) 의 설명 명령에도 같은 이름이 남아 있다.

```
gh repo create my-first-app --public --source=. --push
```

고칠 것 — 두 곳 다 실제 이름으로 통일한다.

이 폴더를 깃허브 저장소로 만들어서 올려 줘. 저장소 이름은 `Study01_MNIST_mac`, 공개로.

B-3. 권한 모드 표의 번호가 1, 2, 3, ✓, 5 다 (이전 B-3)

📍 위치 — 3.2 ▶ 3.2.2 ▶ 3.2.2.2 ▶ d) 잦은 질문을 피하기 위해 클로드의 권한 모드를 변경

🔍 검색어 — 흔히 "YOLO 모드"로 불리는

고칠 곳 — 표의 네 번째 줄 번호 칸

네 번째 줄의 번호 칸에 4 대신 ✓ 가 들어 있다. 옆의 드롭다운 그림(그림 045)에서 자동이 현재 선택돼 체크로 표시된 것을 그대로 옮긴 결과다. 그림을 다시 확인했고 원인이 맞다.

학생은 "4번은 어디 갔지?" 하고 멈춘다. 표 안의 설명문에도 "현재 이미지에서 선택된 모드입니다" 가 들어 있어 번호 칸까지 체크로 바꿀 이유가 없다.

고칠 것 — 번호 칸은 4 로 적고, 선택 여부는 설명문(이미 있다)이나 배경색으로만 표시한다.

B-4. 학습에 몇 분 걸리는지 알려주지 않는다 (이전 B-4)

📍 위치 — 3.2 ▶ 3.2.2 ▶ 3.2.2.2 ▶ f) 손글씨 인식 앱 작성 진행 ▶ f.2) 프롬프트 입력하여 손글씨 인식

앱 작성 진행 (또는 g) 바로 앞)

🔍 검색어 — f.2.3) 실제 적용한 프롬프트

고칠 곳 — f.2.3 뒤에 한 줄 추가

CNN 5 에포크 학습은 맥에서 수 분 걸린다. 그동안 화면에 진행 표시가 거의 없다.

1학년은 멈춘 줄 알고 중단한다. 중단하면 `mnist_cnn.pt` 가 안 생기고, 이후 전부 실패한다. 이 지점은 A-신1 과 함께 실습 성패를 가르는 자리에 있다.

고칠 것 — 한 줄 넣는다. 아래 숫자는 교수님 맥에서 실제로 걸린 시간으로 바꿔 주셔야 한다(아래 6절 참고).

학습에 ○○분 걸린다. 화면이 멈춘 것처럼 보여도 끄지 않는다. 맥 사양에 따라 다르다.

B-5. "폴더에서 새로운 터미널 열기" 가 기본 메뉴가 아니다 (이전 B-5)

📍 위치 — 3.1 ▶ 3.1.1 ▶ 3.1.1.1 (Homebrew 활용한) 맥 설치 방법 ▶ a) 터미널 열기
🔍 검색어 — 단축메뉴로 폴더에서 새로운 터미널 열기 클릭
고칠 곳 — 본문 한 줄 아래

그림 002 의 서비스 메뉴는 이미 활성화된 상태다. 맥 기본값은 꺼져 있다. 게다가 그림에는 반디집으로 압축 풀기, (이름).7z로 압축하기 항목도 함께 있어 학생 화면과 눈에 띄게 다르다. 같은 메뉴가 나오는 그림 064 에서도 마찬가지다.

고칠 것 — 활성화 방법을 덧붙이거나, 더 확실한 대안을 함께 준다.

이 메뉴가 안 보이면 시스템 설정 → 키보드 → 키보드 단축키 → 서비스 → 파일 및 폴더 에서
폴더에서 새로운 터미널 열기 를 켜다.

또는 터미널을 먼저 연 뒤 cd 를 입력하고 폴더를 터미널 창으로 끌어다 놓으면 경로가 자동으로 입력된다.

B-6. 삭제 → 설치 → 갱신 순서다 (이전 B-6)

📍 위치 ① — 3.1.1.1 ▶ c) 클로드 데스크톱 앱 삭제/설치/갱신 방법
📍 위치 ② — 3.1.1.2 ▶ d) 클로드 데스크톱 앱 삭제/설치/갱신 방법
🔍 검색어 — 기존 클로드 데스크톱 앱 삭제 방법
고칠 곳 — 하위 토글 c.1 ~ c.3, d.1 ~ d.3 의 순서

처음 설치하는 학생에게 "기존 앱 삭제 방법"이 맨 앞에 나온다. 지을 것이 없는 학생은 혼란스럽다. 절 제목 자체도 삭제/설치/갱신 순이다.

고칠 것 — 설치를 맨 앞에 두고, 갱신과 삭제는 뒤에 "이미 깔려 있다면" 이라는 조건과 함께 접어 둔다. 제목도 설치/갱신/삭제 로 바꾼다.

B-7. 3.1 예시 폴더와 3.2 실습 폴더가 다르다 (이전 B-7, 부분 완화)

📍 위치 ① — 3.1.1.1 ▶ d) 실행 ▶ 코드 모드 (Code → 새로 생성 → 실행환경(로컬) → 프로젝트 또는 폴더(3장 손글씨 앱))
📍 위치 ② — 3.2 ▶ 3.2.2 ▶ 3.2.2.1 study01_MNIST(Mac) 폴더 생성
🔍 검색어 — 프로젝트 또는 폴더(3장 손글씨 앱)
고칠 곳 — 위치 ①의 토글 제목 옆

3.1 코드 모드 설명은 3장 손글씨 앱 폴더, 3.2 실습은 study01_MNIST(Mac) 폴더다. 폴더를 언제 어디에 만드는지 학생 머릿속에서 꼬인다.

이번 판에서 그림 042 가 두 폴더의 상하 관계를 보여 주어 조금 나아졌다. 3장 손글씨 앱 안에 study01_MNIST(Mac) 이 있다는 것이 화면으로 드러난다. 다만 본문에 그 설명이 없어서 그림을 눌러 본 학생만 알게 된다.

고칠 것 — 위치 ①에 한 줄 붙인다.

여기서는 화면을 구경하기 위해 상위 폴더(3장 손글씨 앱)를 열었다. 실습은 3.2 에서 그 안에 만드는 study01_MNIST(Mac) 폴더로 다시 연다.

3.2 새로 확인한 B 등급 10건

B-신1. base44 절의 토글 제목과 내용이 반대다

📍 위치 — 3.5 base44.com 활용 ▶ 3.5.3 개선 모색 ▶ 3.5.3.1 화면 좌측 하단 대화창에서 Discuss 상태로 전환 ▶ b) 데스크 버전 및 웹 버전 분리 확인
🔍 검색어 — 데스크 버전 및 웹 버전 분리 확인
고칠 곳 — 토글 제목

토글 제목은 "분리 확인" 인데, 안의 그림(그림 107)에서 base44 가 하는 말은 분리하지 않았다는 것이다.

현재 제공해 드린 앱은 '반응형 웹 애플리케이션(Responsive Web App)' 방식으로 구현되어 있습니다. ... 따라서 기술적으로 두 개의 별도 앱으로 분리하는 것보다, 하나의 앱이 모든 환경에서 최적화되도록 구현하는 것이 훨씬 더 안정적이고 효율적인 선택이었습니다.

즉 사용자가 요청한 것(별도 앱 두 개)을 AI 가 이행하지 않고, 하지 않은 이유를 설득한 장면이다. 제목만 읽고 넘어가는 학생은 정반대로 이해한다.

이건 사실 이 절에서 가장 가르칠 만한 장면이기도 하다. 3.2, 3.3 의 클로드는 요청대로 두 버전을 나눴는데 base44 는 안 나눴다. 같은 프롬프트에 도구마다 다르게 답한다는 것이 눈으로 보인다.

고칠 것 — 제목을 사실대로 바꾸고 한 줄 붙인다.

b) 요청한 "데스크톱 버전과 웹 버전 분리"를 base44 는 하지 않았다

반응형 웹 앱 하나로 만들고 그 편이 낫다고 답했다. 요청을 그대로 수행하지 않고 대안을 제시하는 것도 AI 의 흔한 행동이다. 3.3 에서 클로드는 같은 요청에 두 폴더로 나눠 주었다. 도구에 따라 답이 다르다.

B-신2. base44 결과가 실제로 많이 틀리는데 한 줄로만 처리했다

📍 위치 — 3.5 base44.com 활용 ▶ 3.5.3 개선 모색 ▶ 3.5.3.5 인식 정확도는 개선이 필요한 상태 (토글이 아닌 항목)

🔍 **검색어** — 인식 정확도는 개선이 필요한 상태

고칠 곳 — 이 항목을 절로 키운다

그림을 세어 보면 상태가 한 줄로 넘길 수준이 아니다. 게시된 앱 화면(그림 115)의 세션 기록 7건 중 3건이 오답이다.

그린 숫자	AI 의 답	신뢰도
9	4	86%
8	2	82%
7	2	57%
7	2	41%
4	4	98%
3	3	100%
2	2	100%

개선 작업 뒤 화면(그림 111)에서도 8건 중 5건이 틀렸다. 그리고 개선 전 화면(그림 104)에서는 분명한 8 을 "3" 이라고 신뢰도 100.0% 로 답한다.

같은 장 앞부분에서 만든 앱은 98.0% 다. 정확도가 30배 넘게 차이 나는 두 결과가 한 문서 안에 있는데, 그 대조를 문서가 하지 않는다. 학생 입장에서는 "3.5 도 그냥 되는구나" 로 읽고 넘어간다.

한 가지 더. 그림 105 에서 base44 는 스스로 이렇게 말한다.

네, 현재 프리뷰에서 보시는 결과물이 요청하신 '손글씨 숫자 인식기'의 완성된 기능 버전입니다.

틀린 답을 내면서 완성됐다고 말한다. 3.3 의 「9개 미흡한 점」 표가 가르치는 것("검증 통과는 옳다가 아니라 시험해 본 범위 안에서는 옳다")과 정확히 같은 교훈이 여기에 한 번 더 있다.

고칠 것 — 항목을 절로 키우고 대조를 넣는다. 아래를 그대로 옮겨 쓸 수 있다.

3.5.3.5 정확도를 두 앱으로 비교해 보기

	3.2, 3.3 에서 만든 앱	3.5 base44 앱
만드는 방법	클로드에게 프롬프트로 요청	base44 에 프롬프트로 요청
모델	PyTorch CNN 을 직접 학습	TensorFlow.js MNIST
전처리	여백 제거 → 20x20 → 무게중심 정렬	바운딩 박스 정규화 + 가우시안 블러
검증	200장으로 자동 대조	없음

	3.2, 3.3 에서 만든 앱	3.5 base44 앱
실제 정확도	98.0% (196/200)	게시본 7건 중 3건 오답

왜 이렇게 차이가 날까. 3.3에서는 검증 페이지를 따로 만들어 200장을 자동으로 대조했다. base44에서는 그 장치를 만들지 않았고, 그래서 "완성됐습니다"라는 AI의 말을 확인할 방법이 없었다.

base44는 그림 105에서 스스로 "완성된 기능 버전입니다"라고 답했다. 그 말을 한 시점에 앱은 8을 3으로, 9를 4로 읽고 있었다.

이 장에서 가져갈 것은 이것이다. AI가 "완성됐다"고 말해도 그것은 확인이 아니다. 확인은 사람이 기준을 정하고 재 봐야 생긴다.

B-신3. "교재 예제는 Keras"라는 주장이 근거 없이 다시 들어왔다

📍 위치 ① — 3.2.2.2 ▶ f) 손글씨 인식 앱 작성 진행 ▶ f.1) TensorFlow vs. PyTorch ▶ f.1.2) 같은 모델을 두 방식으로 비교 (– Keras (교재) 라는 라벨)

📍 위치 ② — 같은 f.1) ▶ f.1.4) PyTorch를 채택해서 진행한 이유

🔍 검색어 — 따라서 Keras 채택이 불가했고

고칠 곳 — 두 곳의 "교재" 표현

f.1.2)는 두 코드를 나란히 놓으면서 이렇게 라벨을 붙였다.

- PyTorch (강의 노트)
- Keras (교재)

f.1.4)는 이렇게 잇는다.

따라서 Keras 채택이 불가했고, PyTorch를 채택하여 실습을 진행하게 됨

이전 검토서에서 이 주장을 명시적으로 내렸던 것인데 다시 들어왔다. 근거를 다시 적는다. 교재 3장 강의 슬라이드 88장 전체를 확인했을 때 Keras, TensorFlow, PyTorch가 한 번도 나오지 않는다. 교재는 프레임워크를 지정하지 않고 클로드에게 맡긴다.

주장의 출처는 그림 051 안의 클로드 발언으로 보인다.

Python 3.14뿐이고 TensorFlow는 3.14용 휠이 없어 설치 불가입니다. (책 예제는 보통 Keras를 쓰지만 이 환경에선 안 됩니다.)

이것은 클로드의 추측이고, 검증하지 않은 채 본문 라벨로 승격됐다. 저자가 『모두의 딥러닝』(Keras 기반)을 쓴 조태호라서 생긴 연상으로 보인다.

교재 본문을 직접 확인했다 — 교재는 Keras 를 쓰지 않는다

한빛+ 전자책(399쪽)에서 전문 검색으로 대조했다. 세 프레임워크 이름이 전부 96쪽 한 문장에 한 번씩만 나오고, 그 밖에는 어디에도 없다.

검색어	결과
케라스	96쪽 1건
텐서플로	96쪽 1건 (같은 문장)
파이토치	96쪽 1건 (같은 문장)
Sequential	313, 314쪽. 전부 sequential-thinking MCP 이야기다. Keras 모델 코드가 아니다

그 한 문장은 이렇게 시작한다.

"과거에는 이런 프로그램을 만들기 위해 텐서플로나 케라스, 파이토치 같은 프레임워크와..."
— 조태호, 『혼자 공부하는 바이브 코딩 with 클로드 코드』 96쪽

"과거에는" 으로 시작하는 대비 문장이다. 예전에는 이런 것들을 직접 다뤄야 했다는 뜻이고, 세 이름을 나란히 한 번 언급할 뿐 어느 하나를 채택하지 않는다. Keras 예제 코드는 책에 없다.

따라서 Keras (교재) 는 사실과 다르다. 이전 검토서에서 이 주장을 내린 판단이 맞았고, 이번에는 슬라이드가 아니라 책 본문으로 확인했다.

고칠 것 두 가지

첫째, Keras 채택이 불가했고 는 교재와 무관하게 그 자체로 부정확하다. 설치가 안 된 것은 TensorFlow 다. Keras 는 그 위에 있는 API 층이고, 이 관계는 바로 위 그림 048 이 정확히 설명하고 있다.

Keras ← 사람이 쓰는 층(고수준 API)
TensorFlow JAX ← 실제 계산 엔진

그림은 층과 엔진을 구분해 놓고, 두 항목 뒤의 문장은 그 구분을 잃었다. 그림과 본문이 어긋난 것이라 교재와 상관없이 고쳐야 한다.

따라서 TensorFlow 설치가 불가했고(Keras 는 TensorFlow 위에서 도는 층이라 함께 쓸 수 없음), PyTorch 를 채택하여 실습을 진행하게 됨

둘째, **Keras (교재)** 라벨은 **Keras (비교용)** 으로 바꾼다. 한 단어다. 이 표의 목적은 같은 모델을 두 문법으로 견주어 보는 것이지 출처를 밝히는 것이 아니므로, 라벨에서 교재를 떼도 잃는 것이 없다.

한 줄 더 넣을 값이 있다. 교재 96쪽이 세 프레임워크를 "과거에는 직접 다뤄야 했던 것" 으로 묶어 놓은 것과도 잘 맞는다.

교재는 어떤 프레임워크를 쓸지 지정하지 않는다. 그래서 학생마다 다른 결과가 나올 수 있고, 이 수업에서는 프롬프트로 PyTorch 를 지정해 결과를 맞춘다.

이 한 줄이 A-3 처방의 핵심이었는데 현재 판에는 빠져 있다. 지금은 "교수님 맥에는 3.14 만 있었다" 라는 사정 설명으로 끝나고, 학생이 무엇을 해야 하는지가 **f.2.2** 로 흩어져 있다.

B-신4. 저장소에 **data/** 를 올린 화면과, 학생이 실제로 받는 저장소가 다르다

📌 위치 ① — 3.3.3.8 ▶ a) 깃허브에 올리기 ▶ a.4) 뒤 (그림 076)
📌 위치 ② — 3.3 ▶ 3.3.4 최종 작업 결과가 저장된 깃허브 저장소 ▶ 깃허브 저장소 ... 에서 코드 → Download ZIP 클릭하고 압축을 해제하여 테스트 해보기
🔍 검색어 — Download ZIP
고칠 곳 — 3.3.4 에 한 줄 추가

그림 076 에서 클로드는 이렇게 보고한다.

요청하신 대로 **data/** (63MB), **__pycache__/** , **.DS_Store** , **mnist_cnn.pt** , **손글씨인식.app** 모두 포함

그런데 지금 저장소에는 **data/** 가 없다. 방금 원격 저장소를 조회해 확인했다. 이후에 **MNIST** 원본 63MB 를 **git** 추적에서 제외한다 커밋으로 빠졌고, 그 커밋 메시지는 그림 096 화면에도 보인다.

3.3.4 는 학생에게 **Download ZIP** 으로 받아 테스트해 보라고 안내한다. 받으면 **data/** 가 없으므로, **python3 train.py** 를 돌리면 **MNIST** 를 다시 내려받는다. 그 순간 A-4 의 **SSL** 오류를 만날 수 있다. 학생이 예상하지 못한 자리에서 만나는 것이 문제다.

고칠 것 — 3.3.4 에 한 줄 넣는다.

내려받은 폴더에는 **MNIST** 원본(**desktop_version/data/**)이 들어 있지 않다. 용량이 63MB 라 저장소에서 뺐다. **python3 train.py** 를 돌리면 자동으로 다시 받는다. 이때 **SSL** 오류가 나면 **f.2.4** 를 참고한다.

mnist_cnn.pt (학습된 가중치)는 들어 있으므로, 학습 없이 **python3 app.py** 로 바로 실행해 볼 수 있다.

B-신5. **GitHub Actions** 로 바뀌어야 하는 이유를 적다가 문장이 끊겼다

📍 위치 — 3.3.3.8 ▶ d) GitHub Actions로 바꾸는 것에 관한 사항
🔍 검색어 — 우리 실습에서는
고칠 곳 — 해당 항목

현재 이렇게 적혀 있다.

- index.html 파일이 최상위 폴더에 존재하면 배포가 간단해짐 (GitHub Actions 로 바꾸는 작업 불필요함)
- 우리 실습에서는 GitHub Actions 로 바꾸는 작업이 필요한 이유
(Study01_MNIST_mac/web_version/index.html)

둘째 항목이 제목만 있고 이유가 없다. 괄호 안의 경로가 이유를 암시하지만, 1학년은 그 경로만 보고 "최상위가 아니라서 그렇구나" 까지 연결하지 못한다.

고칠 것 — 문장을 완성한다.

- 우리 실습에서 GitHub Actions 로 바꿔야 하는 이유 — 웹 페이지가 최상위가 아니라 web_version/ 안에 있기 때문이다(Study01_MNIST_mac/web_version/index.html). 기본값인 Deploy from a branch 는 저장소 최상위나 /docs 둘 중에서만 고를 수 있어서, web_version/ 을 가리킬 방법이 없다.

B-신6. 3.1.2 CLI 절의 화면이 서로 다른 사람, 다른 버전의 것이 섞여 있다

📍 위치 — 3.1 클로드 데스크톱 설치 ▶ 3.1.2 클로드 CLI 버전 설치 및 실행 (참고용) ▶ b) 윈도우 ▶ b.2) 클로드 실행
🔍 검색어 — 과금 방식 선택: 클로드 계정 구독 방식이면 1번
고칠 곳 — b.2) 의 그림 여덟 장(그림 027 ~ 034)

여덟 장을 하나씩 확인하니 최소 두 사람의 화면이 섞여 있다.

그림	화면에 보이는 것
027	C:\Users\logistex
030	로그인 계정 tae.h.jo@gmail.com
032	Logged in as **tae.h.jo@gmail.com**
033	Claude Code **v2.0.60** / Opus 4.5 · Claude **Max** / C:\Users**taehj**
034	Claude Code **v2.1.183** / Sonnet 4.6 · Claude **Pro** / logistex63@gmail.com's Organization / C:\Users\logistex

계정이 둘(`tae.h.jo` 와 `logistex63`), 홈 폴더가 둘(`taehj` 와 `logistex`), 버전이 둘(`v2.0.60` 과 `v2.1.183`), 요금제가 둘(Max 와 Pro), 모델이 둘(Opus 4.5 와 Sonnet 4.6)이다.

참고용 절이라 실습에 직접 지장은 없지만, 화면을 하나씩 따라가는 학생은 중간에 계정과 화면이 바뀌는 것을 보고 자기가 뭘 잘못했다고 생각한다. 그리고 제3자의 이메일 주소가 그대로 노출된다.

한 가지 더. 그림 030(승인 화면)은 `claude.ai/oauth/authorize` 인데 그림 031(완료 화면)은 `console.anthropic.com/oauth/code/success` 다. 앞의 그림 029 캡션은 "1번(구독 방식)을 권장" 이라고 하는데, 031 은 2번(API 과금) 경로의 결과 화면이다. 두 경로가 섞였다.

고칠 것 — 셋 중 하나면 된다.

1. 교수님 계정 화면으로 통일해 다시 찍는다(가장 깨끗하다).
2. 절 맨 앞에 "이 절의 일부 화면은 다른 환경에서 찍은 것이라 계정 이름과 버전이 다르게 보인다" 를 붙인다.
3. 참고용이므로 그림을 줄이고 명령과 순서만 남긴다.

어느 쪽을 고르시든 제3자 이메일(`tae.h.jo@gmail.com`)은 가려 주시기를 권한다.

B-신7. 윈도우 설치 화면이 맥을 네트워크 드라이브로 붙인 상태에서 찍었다

📍 위치 — 3.1.1.2 (winget 활용한) 윈도우 설치 방법 및 실행 ▶ b) PowerShell 열기
🔍 검색어 — 탐색기에서 아무 폴더나 선택한 후
고칠 곳 — 그림 021 과 이후 명령 화면

그림 021의 탐색기 경로는 `내 PC > AllFiles on 'Mac' (Z:) > Users` 다. 윈도우에서 맥을 네트워크 드라이브(`Z:`)로 붙여 놓고 그 안에서 찍은 화면이다. 그림 023, 024의 프롬프트도 `PS Z:\Users\logistex>` 다.

학생의 윈도우 PC에는 `Z:` 드라이브도, `AllFiles on 'Mac'` 도 없다. 사이드바에 보이는 `music on 'Mac'` , `NetBackup on ...` , `iCloud on 'Ma...` , `Home on 'Mac'` 도 마찬가지다. 첫 화면부터 자기 것과 다르다.

실무적으로도 문제가 있다. 네트워크 드라이브 경로에서 여는 터미널은 일부 명령이 다르게 동작한다. 학생에게 권할 자리가 아니다.

고칠 것 — 윈도우 PC의 로컬 폴더(예: `C:\Users\사용자이름\Documents`)에서 다시 찍거나, "이 화면은 맥을 네트워크 드라이브로 연결한 상태라 경로가 `Z:` 로 보인다. 여러분은 `C:` 로 나온다" 를 붙인다.

B-신8. 참고용 절에 `sudo chown -R` 과 `rm -rf` 가 경고 없이 들어 있다

📍 위치 — 3.1.2 클로드 CLI 버전 설치 및 실행 (참고용) ▶ c) 맥 ▶ c.1) 클로드 설치
🔍 검색어 — 권한을 현재 사용자
고칠 곳 — 그림 037, 038 아래

그림 037, 038에 이 명령들이 있다.

```
sudo chown -R $(whoami):staff /usr/local/lib/node_modules
rm -rf /usr/local/lib/node_modules/@anthropic-ai/claude-code
```

캡션이 붙어 있어 무엇을 하는지는 읽힌다. 다만 1학년이 `sudo` 와 `rm -rf` 를 처음 만나는 자리다. 경로를 한 글자 잘못 치면 되돌릴 수 없다. 실제로 `sudo chown -R` 은 시스템 디렉터리의 소유권을 바꾸는 명령이다.

권한 오류 자체가 "예전에 설치하면서 root 소유로 만들어졌다" 는 교수님 맥의 사정에서 온 것이라(그림 035 캡션에 그렇게 적혀 있다) 학생 대부분은 겪지도 않는다.

고칠 것 — 그림 앞에 한 줄 붙인다.

아래는 예전 설치 흔적 때문에 권한 오류가 난 경우의 해결 과정이다. 처음 설치하는 학생은 이 단계가 없다. `sudo` 와 `rm -rf` 는 되돌릴 수 없는 명령이라, 같은 오류가 나면 명령을 직접 치기 전에 오류 메시지를 클로드에게 그대로 붙여 넣어 물어보는 편이 안전하다.

B-신9. 권한 모드 표는 다섯 가지를 설명하는데, 화면은 '자동 모드'를 켜고 있다

📍 위치 — 3.2.2.2 ▶ d) 잦은 질문을 피하기 위해 클로드의 권한 모드를 변경
🔍 검색어 — 잦은 질문을 피하기 위해
고칠 곳 — d) 본문

d) 는 다섯 모드를 설명하는 표와 드롭다운 그림(그림 045), 그리고 자동 모드 활성화 대화상자(그림 046)를 나란히 놓는다. 그런데 "어느 것을 고르랴"는 지시가 없다.

게다가 화면끼리도 어긋난다. 그림 043 의 입력창 아래에는 편집 자동 수락 (2번)이 표시돼 있고, 그림 046 은 자동 모드를 활성화할까요? (4번) 대화상자이며, 그림 047 의 입력창 아래에는 자동 이 표시돼 있다. 학생이 어느 시점에 무엇을 눌러야 하는지 알 수 없다.

고칠 것 — 표 뒤에 한 줄 넣는다.

이 수업에서는 4번 자동 을 쓴다. 읽기 전용 명령은 묻지 않고 넘어가고, 파일 삭제처럼 위험한 작업은 여전히 물어본다. 아래 그림처럼 자동 모드 활성화 를 누르면 된다. 5번 권한 무시 는 쓰지 않는다.

B-신10. 개요 그림의 교재 목차와 실제 수업 내용이 다르다는 설명이 절반만 있다

📍 위치 — 3.0 개요
🔍 검색어 — Part 1 부분이 대폭 변경됨
고칠 곳 — 그림 001 아래 항목

그림 001 은 교재 목차 세 덩이를 보여 준다.

Part 1. 클로드 코드 설치	Part 2. 손글씨 인식 프로그램	Part 3. CLAUDE.md 활용
노드JS와 깃 설치	MNIST 데이터세트	프로젝트 설정 파일
터미널 사용법	AI 프로그램 만들기	계층적 구조 관리
클로드 코드 설치 및 초기 설정	배치 파일 생성	웹 버전 확장

본문은 "Part 1 부분이 대폭 변경됨" 이라고만 적는다. 그런데 Part 2 의 배치 파일 생성 도 바뀌었다. 맥에는 .bat 이 없어 손글씨인식.app 으로 대체했고, 그 사실은 3.4.1 에 가서야 나온다.

고칠 것 — 항목을 한 줄 늘린다.

- (클로드 코드 CLI 버전이 아닌) 클로드 데스크톱 버전으로 강의를 진행하면, Part 1 이 대폭 바뀌고 Part 2 의 배치 파일 생성 도 손글씨인식.app 만들기로 바뀜 (자세한 차이는 3.4.1)

4. C 등급 — 다듬으면 좋다

4.1 이전 검토서의 C 등급 5건 (전부 미반영, 둘은 범위가 넓어짐)

C-1. 스킬을 화면에서 쓰는데 설명이 없다 (이전 C-1)

- 📌 위치 ① — 3.3 ▶ 3.3.3 ▶ 3.3.3.5 클로드의 구현 계획 (그림 090)
- 📌 위치 ② — 3.3 ▶ 3.3.3 ▶ 3.3.3.6 클로드의 구현 방식에 관한 질문 (그림 091, 092)
- 🔍 검색어 — 3.3.3.5 클로드의 구현 계획
- 고칠 곳 — 3.3.3.5 앞

그림 090 에 스킬 실행됨 `/superpowers:writing-plans` 가 그대로 찍혀 있다. 그림 092 에는 `subagent-driven-development` 으로 실행합니다 가 있고, 그림 091 은 학생에게 "1. 서버에이전트 방식 (권장) / 2. 인라인 실행" 중 고르라고 한다.

학생이 같은 프롬프트를 넣어도 그 스킬이 설치돼 있지 않으면 이 화면들이 안 나온다. 설계 문서와 계획 문서가 만들어지는 과정 전체가 이 스킬 덕분인데, 문서는 그 존재를 언급하지 않는다.

고칠 것 — 별도로 작성한 [스킬 사용 현황과 설치 안내](#) 문서를 3.3.3 앞에서 링크한다. 그 문서를 노선에도 옮겨 두시면 학생이 바로 볼 수 있다.

C-2. 맥 설치 절의 로그인 화면이 전부 윈도우 화면이다 (이전 C-2, 범위 확대)

📍 위치 — 3.1.1.1 (Homebrew 활용한) 맥 설치 방법 ▶ d) 실행 ▶ 첫 실행을 위한 인증
🔍 검색어 — 신교수는 Google로 계속하기 클릭
고칠 곳 — 그림 009 ~ 014 여섯 장

이전 검토서는 그림 009 한 장만 지적했는데, 이번에 여섯 장을 다 확인하니 자신의 계정으로 로그인, 웹 브라우저에서 로그인, 2단계 인증 완료 세 토글의 그림이 전부 윈도우 화면이다.

그림	근거
009	창 장식이 오른쪽 위 - □ × (맥은 왼쪽 위 빨강, 노랑, 초록)
010 ~ 014	브라우저가 Microsoft Edge. 탭 모양, 주소창, 오른쪽 위 버튼이 전부 윈도우

맥 설치 절인데 맥 화면이 한 장도 없다. 학생은 "내 화면과 다른데?" 를 여섯 번 연속으로 만난다.

고칠 것 — 맥에서 다시 찍거나, 토글 제목에 "(윈도우 화면. 맥도 순서는 같다)" 를 붙인다.

C-3. 정확도 수치가 문서마다 다르다 (이전 C-3)

📍 위치 ① — 3.2.2.2 ▶ f) 손글씨 인식 앱 작성 진행 ▶ f.2) ▶ f.2.3) 실제 적용한 프롬프트 및 클로드 작업 내역 (그림 052)
📍 위치 ② — 3.3.3.7 ▶ b) 구현 완료했고, 깃허브 메인에 병합하여 검증 진행 (그림 095)
🔍 검색어 — 테스트 300장
고칠 곳 — 둘 중 한쪽에 각주

그림 052 는 "앱 예측 경로(280×280 캔버스 → 전처리 → 모델) 정확도 98.0% (테스트 300장)", 그림 095 의 최종 검증 표는 "98.0% (196/200)" 이다. 정확도는 같은데 표본 수가 다르다.

우연히 두 수치가 98.0% 로 같아서 눈에 잘 안 띄지만, 표를 비교하는 학생은 "300장인가 200장인가" 에서 멈춘다.

고칠 것 — 200장 기준으로 통일하거나, 둘이 다른 시점의 다른 검증이라는 점을 한 줄로 밝힌다.

300장은 3.2 시점의 1차 확인이고, 200장은 3.3 에서 웹 버전과 대조하려고 새로 만든 검증 데이터다. 최종 기준은 200장이다.

C-4. 화면 속 개인 사용 기록과 개인 파일이 그대로 보인다 (이전 C-4, 범위 대폭 확대)

이 항목은 범위가 크게 넓어져 아래 5절로 따로 뺐다.

C-5. .app 더블클릭 설정은 다른 기기로 안 따라간다 (이전 C-5)

📍 위치 — 3.2 ▶ 3.2.2 ▶ 3.2.2.5 Finder에서 **app.py** 더블클릭하여 완성된 앱 실행하도록 개선

🔍 검색어 — 개선 결과 테스트를 위하여 Finder에서

고칠 곳 — 항목 아래 한 줄 추가

그림 067 에 클로드가 직접 밝힌 내용이 있는데 본문에는 없다.

이 설정은 **app.py** 파일 하나에만 붙는 확장 속성(xattr)이라, 파일을 다른 곳으로 복사하면 따라가지 않을 수 있습니다. SynologyDrive 동기화 폴더라 다른 기기에서는 다시 지정해야 할 수 있습니다.

학교 실습실과 집을 오가는 학생에게 실제로 일어나는 일이다.

고칠 것 — 본문에 짧게 옮긴다.

이 설정은 **app.py** 파일 하나에만 붙는 표시라, 파일을 USB 나 다른 컴퓨터로 옮기면 따라가지 않는다. 옮긴 곳에서 다시 한 번 지정해야 한다.

4.2 새로 확인한 C 등급 10건

C-신1. 코드 블록의 언어가 전부 **javascript** 로 지정돼 있다

📍 위치 — 문서 전체 (3.1.1.1 b) , 3.1.1.2 c) , 3.2.2.2 a) , f.2.1) , f.2.2) , 3.2.2.6 d) , 3.3.3.1 등)

🔍 검색어 — **brew --version**

고칠 곳 — 각 코드 블록의 언어 설정

셸 명령, PowerShell 명령, 한글 프롬프트가 전부 **javascript** 로 지정돼 있다. 노션이 자바스크립트 문법으로 색을 입히므로 엉뚱한 단어에 색이 붙는다. 일부 블록은 **plain text** 로 돼 있어 문서 안에서도 일관되지 않다.

고칠 것 — 셸 명령은 **Bash** , PowerShell 명령은 **PowerShell** , 프롬프트는 **Plain Text** 로 바꾼다. 급하지 않지만 전부 **Plain Text** 로 통일하기만 해도 지금보다 낫다.

C-신2. 클로드 데스트톱 오타

📍 위치 ① — 3.1.1.1 ▶ c) ▶ **c.1)** 기존 클로드 데스트톱 앱 삭제 방법

📍 위치 ② — 3.1.1.2 ▶ d) ▶ **d.1)** 기존 클로드 데스트톱 앱 삭제 방법

📍 위치 ③ — 3.5.3.1 ▶ **b)** 데스톱 버전 및 웹 버전 분리 확인

🔍 검색어 — **데스트톱**

고칠 곳 — 토글 제목 세 곳

데스트톱 → 데스크톱, 데스톱 → 데스크톱.

C-신3. 맥 CLI 절의 캡션이 윈도우 경로를 가리킨다

📍 위치 — 3.1.2 ▶ c) 맥 ▶ c.2) 클로드 실행
🔍 검색어 — C:\Users\logistex 폴더를 신뢰한다면
고칠 곳 — 그림 039 의 캡션

그림 039 의 캡션은 이렇다.

(터미널에서 "claude" 입력하고 엔터)
("C:\Users\logistex" 폴더를 신뢰한다면 1 입력하고 엔터)

맥 절인데 캡션이 윈도우 경로다. 화면에는 /Users/logistex 로 나온다. 윈도우 절(그림 027)의 캡션을 복사해 온 것으로 보인다.

고칠 것 — "/Users/사용자이름" 으로 바꾼다.

C-신4. "종료하려면 'quit' 입력" 이 정확하지 않다

📍 위치 ① — 3.1.2 ▶ b) 윈도 ▶ b.2) 클로드 실행 (그림 034 캡션)
📍 위치 ② — 3.1.2 ▶ c) 맥 ▶ c.2) 클로드 실행 (그림 040 캡션)
🔍 검색어 — 종료하려면 quit 입력
고칠 곳 — 두 캡션

quit 을 그냥 치면 종료되지 않고 프롬프트로 전송된다. 슬래시 명령이므로 /exit 또는 /quit 이어야 한다.

설치된 CLI 바이너리(v2.1.220)에서 명령 정의를 직접 확인했다. name:"exit", aliases:["quit"] 로 등록돼 있다. 즉 /exit 이 정식 이름이고 /quit 이 별칭이며, 둘 다 슬래시가 필요하다.

고칠 것 — (종료하려면 /exit 입력) 으로 바꾼다.

C-신5. .app 을 다른 폴더로 옮기면 동작하지 않는다는 제약이 그림 안에만 있다

📍 위치 — 3.2 ▶ 3.2.2 ▶ 3.2.2.6 손글씨인식.app 번들로 만드는 추가 개선 ▶ b) 작업 결과 생성된 손글씨인식.app
🔍 검색어 — 작업 결과 생성된
고칠 곳 — b) 또는 c) 아래 한 줄 추가

그림 068 안에 이 내용이 있다.

앱과 `app.py` 는 같은 폴더에 두어야 합니다. `launcher` 가 자기 위치 기준으로 세 단계 위에서 `app.py` 를 찾습니다. 폴더째 옮기는 건 괜찮지만, `.app` 만 따로 `/Applications` 등으로 옮기면 동작하지 않습니다. (독에 끌어다 놓는 건 파일을 옮기는 게 아니라 안전합니다.)

`.app` 을 만들면 응용 프로그램 폴더로 옮기고 싶은 것이 자연스러운 반응이다. 옮기면 안 된다는 것을 본문에서 알려 줘야 한다.

고칠 것 — 한 줄 넣는다.

손글씨인식 `.app` 은 `app.py` 와 같은 폴더에 두어야 한다. 응용 프로그램 폴더로 옮기면 동작하지 않는다. 독에 넣는 것은 파일을 옮기는 것이 아니라 괜찮다(c) 참고).

C-신6. 저장소에 README 가 없다

📍 위치 — 3.3 ▶ 3.3.4 최종 작업 결과가 저장된 깃허브 저장소
🔍 검색어 — 3.3.4
고칠 곳 — 저장소 쪽 (노션 문서가 아니라 깃허브)

그림 096 의 저장소 화면에 **Add a README** 안내가 그대로 보인다. 지금도 없다.

3.3.4 는 학생에게 이 저장소를 받아 테스트해 보라고 한다. 받은 학생이 무엇부터 실행해야 하는지 알 방법이 `CLAUDE.md` 를 여는 것뿐이다. `CLAUDE.md` 는 클로드용 지침이라 사람용 안내로는 조금 어긋난다.

고칠 것 — 저장소에 짧은 `README.md` 를 둔다. 세 줄이면 충분하다.

손글씨 숫자 인식 (3장 실습)

- 데스크톱: ``cd desktop_version && python3 app.py``
- 웹: ``cd web_version && python3 -m http.server 8000`` 후 `http://localhost:8000`
- 배포본: `https://logistex.github.io/Study01_MNIST_mac/`

C-신7. Download ZIP 을 하면 강의 노트 검토 문서까지 함께 받는다

📍 위치 — 3.3 ▶ 3.3.4 최종 작업 결과가 저장된 깃허브 저장소
🔍 검색어 — 압축을 해제하여 테스트 해보기
고칠 곳 — 저장소 쪽 (판단 필요)

현재 저장소 최상위에 `docs/` 가 있고, 그 안에 이 강의 노트를 검토한 문서들(2026-08-07-3장-데스크톱-버전-검토의견.md , 인계 메모 등)이 들어 있다. 학생이 **Download ZIP** 을 누르면 함께 받는다.

교육적으로 나쁘지만은 않다. "선생님도 문서를 검토받는다" 를 보여 주는 자료가 될 수 있다. 다만 의도한 것인지 확인이 필요하다. 인계 메모에는 진행 중인 판단 사항과 미해결 항목이 그대로 적혀 있다.

고칠 것 — 셋 중 하나를 고르신다.

1. 그대로 둔다(의도적 공개).
2. docs/ 를 별도 저장소나 비공개 폴더로 옮긴다.
3. 인계 메모만 뺀다.

C-신8. 3.0 개요의 참고 링크가 개인 NAS 를 가리킨다

📍 위치 — 3.0 개요 ▶ 클로드 코드의 두 버전 ▶ 차별점
🔍 검색어 — 클로드 CLI 및 데스크톱 차이
고칠 곳 — 링크

링크 주소가 `https://greenocean.synology.me:5001/d/s/...` 다. 개인 NAS 의 공유 링크다.

지금은 살아 있다(접속 확인함). 다만 공유 토큰이 만료되거나 NAS 가 꺼지면 학기 중에 조용히 끊긴다. 학생이 학교 방화벽 안에서 :5001 포트로 접속할 수 있는지도 확실하지 않다.

고칠 것 — 같은 내용을 노션 하위 페이지로 옮기거나, 저장소에 두고 깃허브 링크를 건다. 3.0 개요 의 표가 이미 같은 내용을 담고 있으므로 링크를 빼도 손해가 적다.

같은 이유로 3.2.2.2 ▶ h) 의 `chatgpt.com/share/...` 링크도 확인해 두시면 좋다. 공유 대화는 삭제되면 사라진다.

C-신9. base44 절의 토글 제목이 내용을 알려 주지 않는다

📍 위치 — 3.5 ▶ 3.5.2 작업 진행 ▶ 3.5.2.2 base44.com 확인 사항 ▶ a) 확인 사항 1, b) 확인 사항 2, c) 확인 사항 3
🔍 검색어 — 확인 사항 1
고칠 곳 — 세 토글 제목

접힌 상태에서는 무엇을 묻는지 알 수 없다. 그림을 열어 보면 각각 이렇다.

지금 제목	실제 내용
확인 사항 1	손글씨 숫자 인식에 어떤 방식을 원하시나요? → 캔버스에 직접 그리기
확인 사항 2	인식 결과를 어떻게 활용하고 싶으신가요? → 숫자 표시, 신뢰도 표시
확인 사항 3	앱의 대상 사용자는 누구인가요? → 교육 현장(학생/교사)

고칠 것 — 제목에 질문과 고른 답을 넣는다. 3.5 는 학생이 각자 다르게 답할 수 있는 절이라, 교수님이 무엇을 골랐는지가 뒤 화면을 이해하는 열쇠다.

C-신10. 교재 확인 문제 가운데 답이 없어진 것이 있다는 사실이 없다

📍 위치 — 3.4 핵심 정리 ▶ 3.4.1 CLI 버전인 교재의 핵심을 정리한 것인데, 데스크톱 버전인 실제 강의와 다소
다름
🔍 검색어 — 없어진 기능이며
고칠 곳 — 항목 아래 한 줄 추가

3.4.1 은 # 키가 없어진 기능이라고 정확히 밝혔다. 여기까지는 해소됐다. 다만 한 걸음이 더 남았다.

교재 슬라이드 87 의 확인 문제는 이렇다.

'#'의 실제 동작 방식으로 올바른 것을 고르세요.
① 입력한 내용을 즉시 CLAUDE.md에 자동 저장
② 저장 위치를 선택하는 프롬프트를 표시한 후 저장 ← 교재의 정답
③ 현재 세션에만 임시로 규칙을 적용
④ 모든 프로젝트의 CLAUDE.md를 동시에 업데이트

②는 과거에는 맞는 답이었다. 지금은 기능 자체가 없으므로 이 문제에는 정답이 없다. 교재 3장 확인 문제 중 못 쓰게 된 것은 이 하나뿐이고, 나머지(슬라이드 86 의 두 문제, 슬라이드 87 의 계층적 구조 문제)는 확인한 범위에서 그대로 유효하다.

고칠 것 — 3.4.1 의 # 키 항목 아래에 한 줄 붙인다. 시험 출제 때 이 노트를 다시 열어 볼 것이므로, 적어 두면 그때 실수하지 않는다.

교재 확인 문제 중 # 의 동작을 묻는 문제(슬라이드 87)는 지금은 정답이 없다. 그대로 출제하면 학생이 없어진 기능으로 평가받는다.

5. 화면에 그대로 보이는 개인 정보와 개인 파일

(이전 C-4 를 확장한 것이다. 항목이 많아 따로 뺐다.)

학생 전체에게 공개하는 문서라 한 번에 모아 적는다. 급한 것은 없지만, 공개 범위를 한 번 정하고 일괄로 처리하시는 편이 낫다.

그림	위치	보이는 것
008, 015,	3.1.1.1 ▶ d) 실행	최근 항목 목록 — 더위를 피하는 방법, 논문초안 검토, 마케터 포트폴리오 홈페이지, GitHub 저장소 portfolio4marketer 등

그림	위치	보이는 것
016, 017		
018	3.1.1.1 ▶ d) ▶ 협업 모 드	미발표 논문 검토 화면. 결과 파일 목록에 논문초안-신해웅교수-20260806...., 본문에 "제목·목적과 실증 데이터가 반대 방향입니다", "6.1 결론: ... ← 5.1 수치와 불일치" 등 논문의 약점이 그대로 적혀 있다
011, 012	3.1.1.1 ▶ d) ▶ 첫 실행을 위한 인증	실명(신해웅), 이메일(logistex63@gmail.com), 보유 기기(Galaxy Z Fold7 , Apple iPad (9th generation))
019, 043, 081	3.1.1.1 d) , 3.2.2.2 b) , 3.3.3.2	사용 통계 — 세션 47~48개, 메시지 25,364~25,593개, 총 토큰 14.4M~14.5M, War and Peace보다 약 19배 . 왼쪽에 세션 제목 20여 개(260801-f Supabase MCP 설정 등)
064	3.2.2.4 ▶ a)	Finder 목록에 아직 수업하지 않은 뒷장 폴더가 전부 노출 — 4장 할 일 관리 , 5장 퀴즈게임 base44 , 6장 Study_04 , 7장 공감 다이어리 앱 , 8장 Study_06
070	3.2.2.6 ▶ c)	전체 데스크톱 화면. Downloads 목록에 업무 문서 — 1. 정기 주차권...신청 안내.pdf , 붙임 1_평가위...화_매뉴얼.pdf , 2026년_정보...협조_요청.hwp , 2026학년도 2...임면 사항.xls
030, 032, 033	3.1.2 ▶ b.2)	제3자 계정 tae.h.jo@gmail.com , 홈 폴더 C:\Users\taehj (B-신6 참고)
046	3.2.2.2 ▶ d)	전체 경로 /Users/logistex/Library/CloudStorage/SynologyDrive-gDrive/ 연도/yr26/DS26/vibe coding (taehojo)/3장 손글씨 앱/...
098, 103	3.5.2.1 , 3.5.2.3	신해웅's Workspace , What will you build next , 신해웅?

우선순위를 매기면 이렇다.

순 위	대상	이유
1	그림 018	미발표 논문의 내용과 검토 지적이 그대로 보인다. 다른 그림으로 바꾸는 편이 안전하다
2	그림 070	임면 사항, 평가위원 매뉴얼 같은 업무 문서 이름이 보인다. 창을 좁혀 다시 찍거나 Downloads 영역 을 가린다
3	그림 064	뒷장 폴더가 미리 보인다. 학생 호기심에는 좋지만 강의 순서가 깨진다
4	그림 030, 032, 033	제3자 이메일. B-신6 을 처리하면 함께 해결된다
5	그림 019, 043, 081	사용 통계. 지우기 어렵다면 "교수님 화면이라 기록이 많다. 여러분 화면은 비어 있다" 한 줄이면 총 분하다

5번은 한 줄로 끝나므로 지금 바로 하실 수 있다. 넣을 자리는 3.2.2.2 ▶ b) 첫 세션을 열기 위하여 프롬프트 입력하고 엔터 입력 이 좋다. 학생이 처음 이 화면을 만나는 곳이다.

6. 잘 되어 있는 점

이전 검토서에 적은 것 말고, 이번 판에서 새로 좋아진 것을 적는다.

- 3.1.1.1 b) 와 3.1.1.2 c) 의 설치 안내가 매우 좋다. 확인 → 실패 메시지 → 설치 → 확인 이라는 순서가 두 운영체제에서 같고, "암호는 화면에 표시되지 않으니 그냥 입력하고 엔터", "winget 은 앱 설치 관리자 앱에 들어 있어 스토어에서 winget 으로 검색하면 안 나온다" 처럼 실제로 학생이 걸리는 지점만 골라 적었다.
- f.1) TensorFlow vs. PyTorch 절이 새로 생긴 것이 이번 판 최대의 수확이다. 특히 그림 048 의 관계도 (Keras 는 사람이 쓰는 층, TensorFlow / JAX 는 계산 엔진, PyTorch 는 둘이 한 몸)는 1학년이 평생 헛갈리는 것을 그림 한 장으로 정리한다. 그림 049, 050 의 나란한 코드 비교도 좋다. 그림 049 의 PyTorch 코드는 저장소의 desktop_version/model.py 와 정확히 일치한다(대조 확인함).
- 3.4.1 의 처리 방식이 모범적이다. 교재 그림을 지우지 않고 남긴 채 다른 점 세 가지를 옆에 적었다. 학생은 교재도 볼 수 있고 무엇이 왜 다른지도 안다.
- 3.3.2.5 a) 의 # 확인 과정(그림 077, 078)이 이 장에서 교육적으로 가장 값지다. "예전에는 이랬는데 지금은 없어졌다" 를 공식 문서를 찾아 확인하는 과정이 그대로 담겼고, 대체 방법 표까지 있다. 도구가 계속 바뀐다는 것을 실물로 가르치는 자리다.
- 3.3.3.7 c) 의 「9개 미흡한 점」 표와 그 아래 다섯 줄이 여전히 이 장 전체에서 가장 좋다. "검증을 통과했다는 말은 '옳다'가 아니라 '시험해 본 범위 안에서는 옳다'는 의미" 로 맺은 문장이 특히 그렇다.
- 3.3.3.8 a.5) 의 사진첩 비유와 「자주 실수하는 것」 표, "토큰을 만들어 ~/.zshrc 에 넣으라는 인터넷 안내는 따르지 마세요" 경고는 이전 판 그대로 잘 남아 있다.
- 3.5 를 추가하신 판단 자체가 좋다. 같은 문제를 다른 도구로 풀어 보는 절은 이 수업의 가치를 크게 키운다. B-신1, B-신2 만 손보면 아주 좋은 절이 된다.

7. 판단이 필요한 것

항목	내용
B-4 (학습 시간)	교수님 맥에서 5 에포크 학습이 실제로 몇 분 걸렸습니까? 그 값을 넣어야 학생이 안심합니다. 모르시면 <code>python3 train.py</code> 를 한 번 돌려 재 두시면 됩니다
C-3 (정확도 기준)	200장으로 통일할지, 300장(그림 052)과 병기하고 각주를 달지
5절 (개인 정보)	어디까지 지우실지. 최소한 그림 018, 070 은 교체를 권합니다
C-신7 (docs/ 공개)	검토 문서와 인계 메모를 학생이 받는 저장소에 그대로 둘지

항목	내용
B-신2 (base44 절)	제안한 비교 표를 넣으실지. 넣으면 3.5 가 이 장의 마무리로 잘 맞습니다
B-신6 (CLI 절 그림)	다시 찍을지, 안내 문구만 붙일지, 그림을 줄일지

8. 우선순위 제안

수업 시작까지 시간이 한정돼 있다면 이 순서를 권한다.

순 서	항목	걸리는 시간	이유
1	A-신1 (3.2.2.2 a) 프롬프트 교체)	2분	코드 블록 하나. 안 고치면 A-3 을 고친 효과가 사라진다
2	A-신2 (winget 명령 공백)	1분	글자 하나. 그대로는 실행되지 않는다
3	B-2 (my-first-app → Study01_MNIST_mac)	3분	두 곳. 뒤의 모든 링크가 어긋난다
4	B-3 (권한 모드 ☒ → 4), B-신9 (어느 모드를 쓸 지 지시)	5분	같은 절이라 함께 고친다
5	B-1 (부동소수 예시 정정)	10분	그림을 못 고치면 본문에 정정 한 줄이면 된다
6	B-4 (학습 시간), B-5 (서비스 메뉴), C-5 (.app 이동), C-신5 (.app 위치)	각 5분	전부 한 줄 추가다
7	5절 1~2번 (그림 018, 070 교체)	20분	공개 문서라 미루기 어렵다
8	B-신3 (TensorFlow 설치가 불가 로, Keras (비교용) 으로)	5분	두 단어다. 교재 본문을 확인했고 Keras 를 쓰지 않는다. 지금은 그림 048 과 바로 아래 본문도 서로 어긋나 있다
9	B-신1, B-신2 (base44 절)	30분	이 장의 마무리가 크게 좋아진다
10	나머지 B, C	학기 중 순차	급하지 않다

1~6번을 합쳐 30분이면 끝난다. 여기까지만 해도 실습이 끊기는 자리는 없어진다.

9. 검토 방법

확인한 것

- 노션 문서를 API 로 내려받아 본문 전체(그림 주소를 뺀 40,164자)를 읽었다.

- 삽입된 그림 118장을 모두 확인했다. 캡션이 있는 것은 14장뿐이라, 나머지 104장은 그림을 보지 않으면 평가할 수 없는 상태였다.
- 이전 판과 그림을 대조했다. 새로 들어온 그림은 5장(048, 049, 050, 071, 097), 빠진 그림은 2장이다. 나머지는 같은 파일이고 번호만 밀렸다. 이전 검토서의 그림 번호와 이번 번호의 대응을 계산해 두었다.
- 부동소수 예시(B-1)는 파이썬으로 다시 실행해 대조했다. $20 * (180/20)$ 은 180.0 , $19 * (21/19)$ 는 21.000000000000004 다.
- 그림 049의 PyTorch 코드는 저장소의 `desktop_version/model.py` 와 한 줄씩 대조해 일치를 확인했다.
- `data/` 가 저장소에 없다는 것(B-신4)은 깃허브 API로 원격 저장소를 직접 조회해 확인했다. 저장소 크기는 약 27MB 다.
- `/exit` 과 `/quit` (C-신4)은 설치된 CLI 바이너리(v2.1.220)안의 명령 정의에서 `name:"exit", aliases: ["quit"]` 를 찾아 확인했다.
- 학생에게 주는 링크 세 개(배포 주소, base44 게시 주소, 개인 NAS 참고 링크)는 실제로 요청해 셋 다 응답하는 것을 확인했다.
- `logistex/digitsight-ai` 저장소는 그림 117, 118에 Private로 찍혀 있으나 지금은 공개 상태다. 그림과 현재 상태가 다르다는 점만 밝혀 둔다(그래서 이 항목은 지적으로 올리지 않았다).

확인하지 못한 것

- 학생 맥에서 처음부터 따라 해 보지는 않았다. 설치 절의 판단은 "기본으로 설치돼 있지 않다"는 사실에 근거한 추론이다.
- 윈도우 항목은 이 맥에서 실행해 볼 수 없었다. A-신2의 `Invoke-WebRequest` 는 PowerShell 문법상 공백이 필요하다는 점에 근거한 것이고, 실제로 윈도우에서 돌려 보지는 않았다. 수업 전에 윈도우 PC 한 대에서 3.1.1.2를 처음부터 밟아 보시기를 권한다.
- 교재는 강의 슬라이드(88장)와 본문(한빛+ 전자책, 399쪽)을 모두 확인했다. 슬라이드에는 프레임워크 언급이 0건이고, 본문에는 96쪽 한 문장에 세 이름이 한 번씩 나올 뿐이다(Sequential 검색 결과도 Keras 코드가 아니라 `sequential-thinking` MCP 언급이었다). B-신3은 추론이 아니라 이 대조에 근거한다.
- base44 앱을 직접 조작해 보지는 않았다. B-신2의 정확도 수치는 그림 104, 111, 115의 세션 기록을 읽은 것이다.
- 학습 소요 시간(B-4)은 재 보지 않았다.